Q6. A Creative Agent that Suggests Product Modifications (GA + Semi-Supervised Learning)

> **Question.** Design an intelligent agent that suggests creative modifications to an existing product, using a genetic algorithm and semi-supervised learning. What changes to PEAS and the agent structure make it creative? Take a multi-agent scenario where co-operative creativity is shown.

1. The idea in plain words

A company has one product and only a few past launch results. The agent does two jobs. First, it **learns what sells** from the few labelled launches plus many unlabelled ideas (semi-supervised learning). Second, it **invents variants** with a genetic algorithm (GA) and uses what it learned to score them. Creativity comes from the fitness: it rewards *predicted success + novelty − cost*, not success alone. The agent hands back a short list of options. It does not give one verdict.

2. Assumptions

- The product is a **smart water bottle**. An idea is 8 genes: capacity, material, sensor, display, app, power, shape, price (8,748 possible variants).
- Today's bottle: 500 ml, plastic, no sensor, no display, no app, coin cell, classic shape, budget price.
- We know the outcome of **40 past launches** (20 hits, 20 flops) and have **400 unlabelled** new ideas. A hidden "market" decides real success. The agent never sees it. We use it only to test the final ideas. About 40% of random ideas would really succeed.
- Novelty = how many genes differ from the nearest bottle in today's product line.

3. Design: GA + semi-supervised learning

Part	What it does here
Chromosome	8 genes, each picks an option (e.g. sensor = intake tracker)
Semi-supervised learner	Self-training with logistic regression. It labels its own confident guesses and learns again, so the **learn set evolves**
Fitness	$w_1 \cdot P(\text{success}) + w_2 \cdot \text{novelty} - w_3 \cdot \text{cost}$
Operators	Tournament selection, uniform crossover (mix genes of two parents), mutation (15% per gene), elitism (keep the 2 best)
Exit	30 generations, then show the top 3 ideas. Real market results go back in as labels

4. What changes in PEAS and the agent structure?

A normal agent maximises one number in a fixed world. A creative agent changes all four parts of PEAS and adds two blocks to the agent structure.

	Ordinary product agent	Creative agent (the change)
Performance	Predicted sales	Success **+ novelty + usefulness − cost**, checked by a **creativity calibrator**
Environment	Our own sales data	**Open** and dynamic: competitors, customer reviews, IoT usage data, ideas from other products
Actuators	Report one best design	**Generate** variants, combine features, and present **ranked choices**
Sensors	Sales figures	Also senses **previous performance and competing performance** and user reactions

In the agent structure we add a **creativity evaluator** (the novelty score) and a **conceptual space expander** (crossover and mutation create gene combinations never launched). The learning agent's *problem generator* is the GA, and its *critic* is the market test.

5. Step by step

1. Encode today's bottle and the 40 labelled launches as gene vectors.
2. Train the self-training model on labelled + unlabelled ideas to get P(success) for any variant.
3. Give each agent its own island of 20 random ideas and its own fitness weights.
4. Evolve: select, cross over, mutate. Every 5 generations the agents swap their best ideas (migration).
5. After 30 generations, show the top 3 ideas. Test them in the market and add the results as true labels.

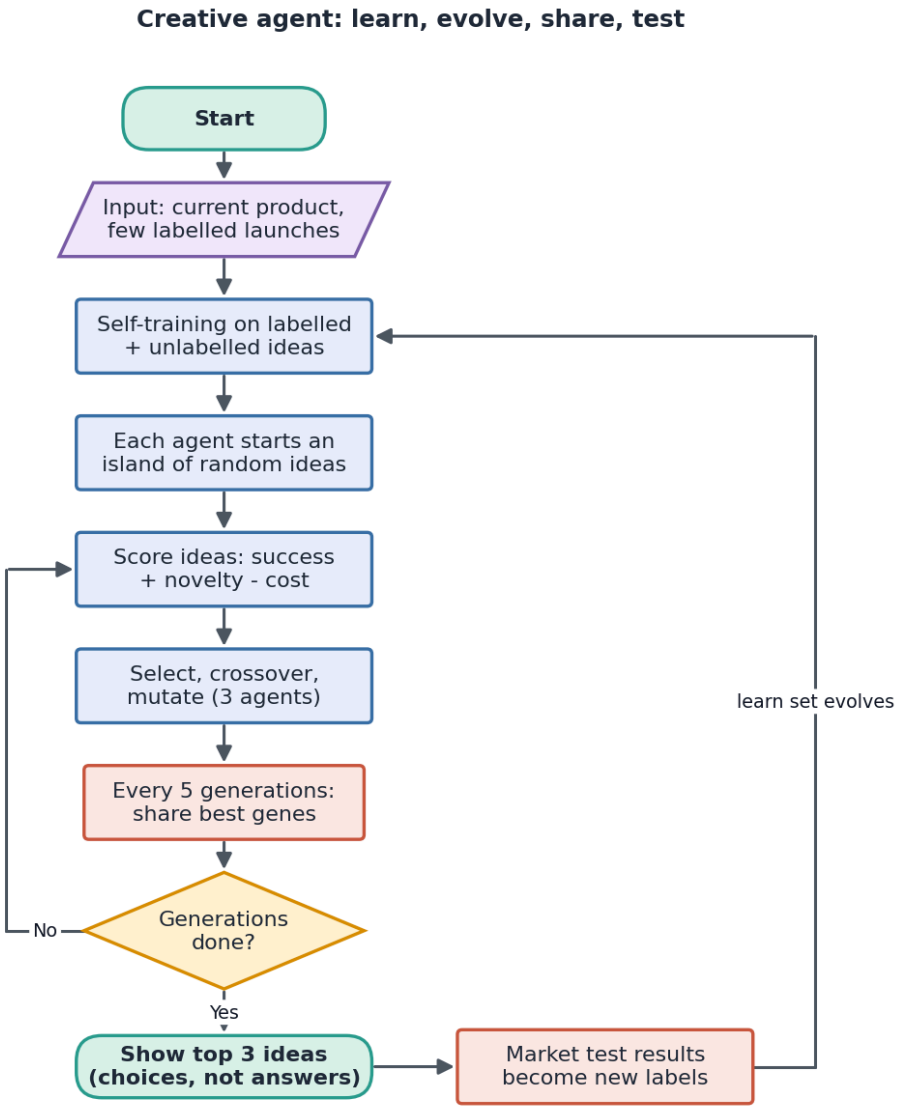


Figure 1. Flowchart of the creative agent. The loop on the right closes the cycle: market results become new labels.

6. Pseudocode

```
model ← SelfTraining(labelled launches, unlabelled ideas)      # semi-supervised
fitness_a(v) ← w1_a·P_model(v) + w2_a·novelty(v) − w3_a·cost(v)   # per agent a
island[a] ← 20 random ideas, for each agent a in {designer, critic, cost}
for gen = 1 to 30:
    for each agent a:
        keep 2 best ideas (elitism)
        repeat until island full:
            p1, p2 ← tournament picks;  child ← uniform_crossover(p1, p2)
            mutate each gene with probability 0.15
    if gen is a multiple of 5: each island receives the best 2 ideas of the others
top3 ← best ideas by shared value;  run market test
add test results to labelled set;  retrain model          # the learn set evolves
```

7. Multi-agent co-operative creativity

Three agents look at the same bottle with different eyes. All three share the same learned model, and they differ only in fitness weights (success, novelty, cost):

Agent	Cares about	Weights (P, novelty, cost)
Designer	bold, new ideas	0.5, 1.0, 0.0
Usability critic	will users like it?	1.0, 0.1, 0.1
Cost agent	cheap to build	0.6, 0.0, 0.8

In the **co-operative** run the islands trade their best genes every 5 generations. Results, averaged over 10 random seeds:

Setup	Mean best shared value	Ideas that really succeed
Designer alone	1.049	90%
Usability critic alone	1.121	40%
Cost agent alone	1.060	0%
Co-operative (islands + migration)	**1.117**	**70%**

> **Honest reading.** The co-operative team is *not* the top scorer on the model's own value (the critic is 1.121 vs 1.117). It does **better on real market success than the critic (70% vs 40%) and the cost agent (0%)**, and its value is far above the designer's. Its main gain is robustness: it avoids the weak spots of any single agent. The designer's 90% is a lucky fit with this made-up market, and it loses on cost and model value.

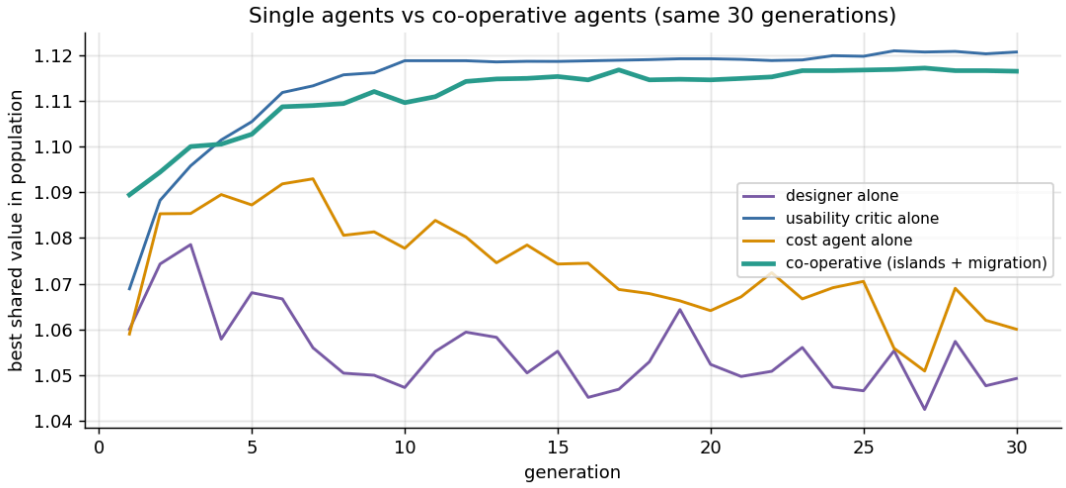


Figure 2. Best shared value per generation (average of 10 seeds). Co-operative agents climb as high as the best single agent.

8. Worked example (real run)

Self-training labelled 305 of the 400 unlabelled ideas itself, so the learn set grew from 40 to 345. Test accuracy stayed about the same (0.73 supervised, 0.72 self-trained), so with so few labels the pseudo-labels did not add real accuracy. What did help was real feedback: after the 3 best ideas were tested, accuracy rose to **0.76**. The top 3 ideas the co-operative agents suggested:

#	Suggested bottle	Value	Market test
1	1 L, plastic, intake tracker, LED ring, reminder app, coin cell, modular cap, premium	1.110	success
2	1 L, plastic, temp sensor, LED ring, reminder app, coin cell, modular cap, premium	1.081	success
3	750 ml, plastic, no sensor, no display, reminder app, coin cell, modular cap, mid	1.067	flop

Two of three succeeded. The ideas are creative: none was in the product line, and the "modular cap + app + sensor" combination is new. The flop shows why the agent returns **choices, not answers**: a human or a test still decides.

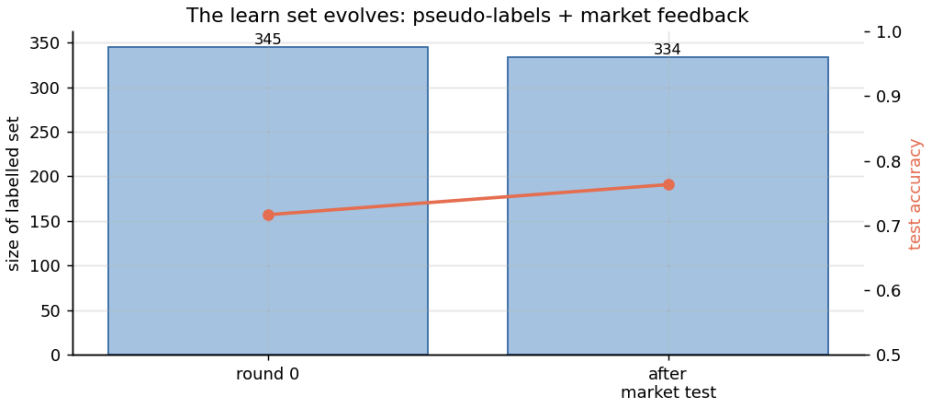


Figure 3. Labelled + pseudo-labelled set size (bars) and test accuracy (line). Accuracy moves from 0.72 to 0.76 once real market labels arrive.

9. Code and real run

File Edit View Run Terminal Help — q06_code.py - Q06_Creative_Product_Agent_GA_SSL - Python IDE

q06_code.py

```python
def evolve(islands: dict, gens: int = 30, migrate: bool = False) -> tuple:
    pops = {name: [random_variant() for _ in range(20)] for name in islands}
    history = []
    for gen in range(gens):
        for name, w in islands.items():
            pop = sorted(pops[name], key=lambda v: -agent_fitness(v, w))
            children = pop[:2]                                  # elitism
            while len(children) < len(pop):
                p1, p2 = (max(random.sample(pop, 3), key=lambda v: agent_fitness(v, w))
                          for _ in range(2))                    # tournament selection
                child = tuple(a if random.random() < 0.5 else b for a, b in zip(p1, p2))
                child = tuple(random.randrange(n) if random.random() < 0.15 else g
                              for n, g in zip(SIZES, child))    # mutation
                children.append(child)
            pops[name] = children
        if migrate and gen % 5 == 4:                            # share best genes
            best = {n: sorted(p, key=lambda v: -agent_fitness(v, islands[n]))[:2]
                    for n, p in pops.items()}
            for n in pops:
                pops[n] = pops[n][:-4] + [v for m in pops if m != n for v in best[m]]
        history.append(max(value(v) for p in pops.values() for v in p))
    return sorted({v for p in pops.values() for v in p}, key=value, reverse=True), history
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Users\USER\questions\Answers\Q06_Creative_Product_Agent_GA_SSL> python q06_code.py
Q6  Creative product agent: GA + semi-supervised learning (smart water bottle)
  today's bottle: 500 ml, plastic, no sensor, no display, no app, coin cell, classic, budget
  share of ideas that really succeed: 40%
  supervised only (40 labels): test accuracy 0.73
  round | labelled | pseudo-labelled | test accuracy  (self-training, 400 unlabelled)
      0 |       40 |             305 | 0.72
GA AGENTS (20 ideas per agent, 30 generations, average of 10 seeds)
  designer         | mean best value 1.049 | really succeed 90%
  usability critic | mean best value 1.121 | really succeed 40%
  cost agent       | mean best value 1.060 | really succeed 0%
  co-operative     | mean best value 1.117 | really succeed 70%
TOP 3 MODIFICATIONS (choices, not answers) + simulated market test
  1. 1 L, plastic, intake tracker, LED ring, reminder app, coin cell, modular cap, premium
     value 1.110 | predicted P 1.00 | market test: success
  2. 1 L, plastic, temp sensor, LED ring, reminder app, coin cell, modular cap, premium
     value 1.081 | predicted P 1.00 | market test: success
  3. 750 ml, plastic, no sensor, no display, reminder app, coin cell, modular cap, mid
     value 1.067 | predicted P 1.00 | market test: flop
  feedback: 3 launch results added as true labels -> accuracy 0.76
charts saved to images/
PS C:\Users\USER\questions\Answers\Q06_Creative_Product_Agent_GA_SSL>
```

main — Ln 154, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

Figure 4. The GA function (lines 133-154) and the real terminal output of the run.

10. Summary

> Semi-supervised learning supplies a success score from very few labels, and the GA turns it into new ideas. Adding novelty and cost to the fitness, sensing the market, and generating ranked choices make the agent creative. Three agents with different goals, sharing genes, give a team that is nearly as good as the best single agent on the model's value and much more dependable in the real market. The honest limit: with 40 labels, pseudo-labels did not raise accuracy, and real feedback did.

Files in this folder: `q06_code.py` (source), `q06_code.ipynb` (Colab/Jupyter version, already run), `q06_output.txt` (console output), `images/` (all figures).